

## TECHNICAL PROJECT FOLLOW-UP

# Meeting Follow-Up

OPF/FOR performance, fairness formulation, and development coordination

**Participants:** Tulio Coelho Coura and Ignacio Pérez Ortiz

**Date:** August 7, 2026, 4:11 PM

**Duration:** 1 hour 17 minutes

**Purpose:** Review implementation progress, research framing, and next development steps

**Executive summary.** Nacho showed that the OPF/FOR workflow can run much faster after removing repeated work and using Claude Code to monitor jobs. The team will now test two parallel-computing options, improve the fairness method so it respects each battery's real capability, and simplify the paper's framework. The main technical conclusion was that the TSO selects one point from the FOR; a fair OPF then assigns the required response to prosumers. DOEs describe a different use case and may not be needed in the core method.

## Key takeaways

- **The workflow is much faster.** Removing repeated data loads and an unused second solve cut runtime by about 75%. Reusing the model matrix added a further improvement of roughly 25%.
- **Both parallel-computing options are worth testing.** One option runs the directional solves together on a supercomputer. The other runs different transformers in separate AIMMS sessions. The target is a complete run within 24 hours.
- **Fairness should not mean equal power from every battery.** Equal sharing can be infeasible or inefficient. The method should reflect each battery's available capability and may also consider how revenue is shared.
- **The FOR and DOE solve different problems.** The TSO chooses one P-Q point from the FOR. A fair OPF then calculates the prosumer setpoints. A DOE describes a prosumer's safe operating range, so it does not fit the service-dispatch step.
- **Development will follow a shared schedule.** Tulio and Nacho will assign development days, merge changes daily, and start each hand-off from the latest main branch.

## Main points discussed

### 1. Runtime optimization and automation

- **Monitoring and automation:** Claude Code can monitor a AIMMS job from PowerShell, report its progress, estimate the remaining time, and send status notifications. It can also run several cases in sequence without manual restarts.
- **Repeated work was removed:** The code had been reloading unchanged data, running a second solve that was not used, and rebuilding the same matrix for every case. These tasks were moved, removed, or reused.
- **Reported improvement:** The first changes saved about 75% of the runtime. Matrix reuse then produced roughly 25% additional improvement. The benefit should be even clearer on larger grids and the full feeder.
- **Solver settings also helped:** CPLEX resolved the direction infeasibilities seen with Gurobi. When using the barrier method, removing the warm start reduced the direction-solve time by about half.

### 2. Parallelization and scalability

- **Supercomputer approach:** Run the 12 independent directional OPFs at the same time, then move to the next time slot. The team still needs to confirm how jobs will communicate and how many nodes or cores are available.
- **License question:** Tulio's supercomputer uses Linux, but the current AIMMS license is for Windows. The team must confirm whether a Linux license is available, how long it lasts, and what it costs. The US\$500 figure mentioned in the meeting was not verified.
- **Local alternative:** Use the available AIMMS sessions to run different transformers at the same time, then combine the results at the network level. This may be easier to implement than the supercomputer approach.
- **Test plan:** Benchmark TR9 and the full feeder with 12, 24, and 36 directions. The goal is to improve angular resolution while keeping the total runtime below 24 hours.

### 3. Fairness formulation and feasibility

- **Current options:** The “m” parameter selects the no-fairness case or a fairness option for aggregators or prosumers. The results show that stronger fairness rules reduce the size of the FOR.
- **Low-state-of-charge issue:** The proportional-export case ( $m=3$ ) can become infeasible when a battery is close to the 10% charge limit. If one battery cannot participate, the current rule may force the other batteries to remain idle.

- **Next method to test:** Base each battery's share on the power it can actually import or export at that time. This should be more feasible than requiring fixed equal or proportional setpoints.
- **Broader view of fairness:** An efficient technical dispatch may create more total value than equal power sharing. Fairness could then be addressed by sharing the revenue or benefits more fairly among prosumers.

#### 4. FOR, disaggregation, and the role of DOEs

- **What the FOR does:** The FOR tells the TSO which P-Q operating points the distribution grid can provide at the connection point.
- **How service is dispatched:** The TSO selects one point from the FOR. The OPF is then solved again with that request fixed, using fairness rules to calculate the required setpoint for each prosumer.
- **Why a DOE is different:** A DOE gives one prosumer a safe range for independent operation. During a requested service, however, prosumers are coordinated to deliver one specific grid-level point.
- **Working conclusion:** The paper should not derive the FOR from DOEs, or DOEs from the FOR, because the two concepts use different assumptions. DOEs may be mentioned for periods without a requested service, but they may not be needed in the main framework.
- **Incentives and limitations:** The paper should explain how prosumers are encouraged to follow the dispatch. Payment for the service should be high enough to cover the value they give up by not using the energy themselves.

#### 5. Repository and development workflow

- **Latest code:** The pull request was merged. Tulio reviewed how to check local differences and pull the latest main branch.
- **Local files:** A clean worktree was created. Any files not stored in Git, including local data, must be moved before the old folder is removed.
- **Remaining procedures:** The current procedures have been tested for TR9. Similar procedures are still needed for the other transformers.
- **Development schedule:** Tulio and Nacho will work on assigned days and merge at the end of each day. The next person will begin from the updated main branch. The schedule will be agreed at Tuesday's meeting.

## Decisions and working agreements

1. **Test both ways to run cases in parallel.** Compare the supercomputer option with running separate transformers in multiple GAMS sessions.
2. **Use the FOR and a fair OPF for service dispatch.** The TSO will request one point from the FOR. The OPF will then calculate a fair and feasible response from the prosumers.
3. **Keep the DOE outside the service step.** Use it only if it adds value for periods without a service request. Confirm the final scope with Luis.
4. **Use a more practical fairness rule.** First reflect each battery's available capability, then consider whether revenue should be shared separately.
5. **Assign development days.** Merge changes daily so that only one current version of the model is in use.

## Action items

The next actions are grouped by owner and expected timing.

| Owner       | Action                                                                                                                 | Timing                    |
|-------------|------------------------------------------------------------------------------------------------------------------------|---------------------------|
| Tulio       | Send Nacho the previous AIMMS support emails and confirm receipt.                                                      | Immediate                 |
| Tulio       | Confirm the Linux AIMMS license options, duration, and cost with support, Luis, and/or the supervisor.                 | Before license decision   |
| Tulio       | Pull the latest main branch, preserve local files, and run the TR9 procedures.                                         | Before Aug. 11 meeting    |
| Tulio       | Create run procedures for the other transformers, if time permits.                                                     | Next development window   |
| Tulio       | Share the fairness notes, including alternatives to equal allocation.                                                  | Next week                 |
| Nacho       | Ask the supercomputer team about the operating system, AIMMS access, available cores, and parallel-job support.        | Next week                 |
| Nacho       | Implement and test capability-aware fairness, especially near the 10% battery charge limit.                            | Next fairness iteration   |
| Nacho       | Avoid repository changes until the development schedule is agreed.                                                     | Through Aug. 11           |
| Both        | Benchmark TR9 and the full feeder with 12, 24, and 36 directions. Record runtime, feasibility, and result consistency. | After environment setup   |
| Both + Luis | Confirm the role of DOEs, the fairness metric, prosumer incentives, and the paper's limitations.                       | Next technical discussion |
| Both        | Agree on development days and the daily merge and hand-off process.                                                    | Aug. 11 meeting           |

## Questions for the next meeting

- Can the team obtain a Linux-compatible AIMMS license? What will it cost, and how long will it last?
- Which parallel-computing option gives the best runtime improvement without changing the results?
- Does capability-aware fairness remove the low-charge infeasibility? Which fairness measure should be reported?
- Should the paper omit DOEs from the main framework or mention them only for periods without a service request?
- How should prosumers be paid so that providing the service is more attractive than self-consumption?
- Which transformer procedures and tests are needed before the full-feeder benchmark?